# SOFTWARE ARCHITECTURE - AN OVERVIEW

SE3030 - SOFTWARE ARCHITECTURE

SEMESTER 1, 2026

SLIIT

# LEARNING OUTCOMES

- After completing this topic, you will be able to,
  - Precisely define what software architecture is.
  - Appreciate the dynamic and the ever-changing nature of software architecture.
  - Understand the expectations of a software architect.
  - Understand the evolution of different architectural styles in the context that they developed.

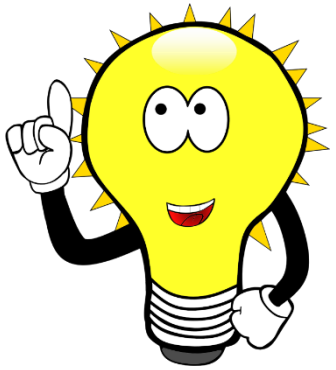
# CONTENTS


- What is Software Architecture?
- Architectural Styles
- Quality Attributes (Architectural characteristics)
- Architectural Decisions
- Design Principles
- What is a Software Architect?
- Expectations of an Architect
- Evolution of Architectural Styles
- References & Further Reading
- Summary

# WHAT IS SOFTWARE ARCHITECTURE?

*"Architecture is about the important stuff…whatever that is."*

Ralph Johnson



# WHAT IS SOFTWARE ARCHITECTURE?

- Software architecture is sometimes defined as,
  - A *plan/ blueprint* for a certain software system.
  - *A Roadmap* for developing a software system.
- But what should this *plan/ blueprint/ roadmap* contain?
  - This is ambiguous.
  - We need a **clearer definition**.

# WHAT IS SOFTWARE ARCHITECTURE?

- *Mark Richards & Neal Ford* in their influential book *"Fundamentals of Software Architecture: An Engineering Approach"* define **software architecture as a combination of**,
  - the structure of the system (**architectural style**),
  - architecture characteristics (**quality attributes**) the system must support,
  - **architecture decisions**
  - and **design principles**

# WHAT IS SOFTWARE ARCHITECTURE?

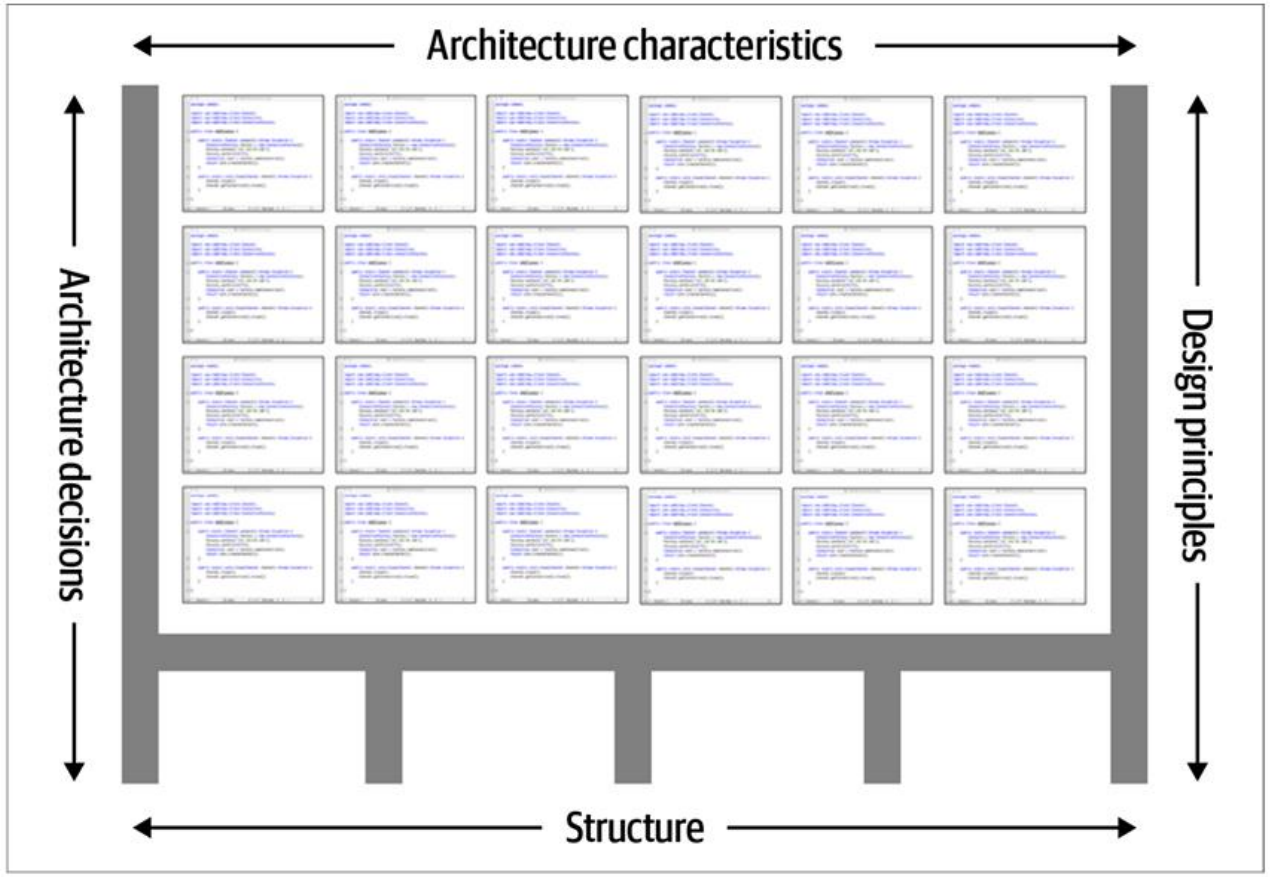



*Figure 1-2. Architecture consists of the structure combined with architecture characteristics ("-ilities"), architecture decisions, and design principles*

# ARCHITECTURAL STYLE

- System structure refers to the chosen **architectural style**(s) (e.g., layered, microservices, microkernel).
- Describing architecture only by its structure is insufficient.
  - Saying “it’s a microservices architecture” explains how the system is organized, not the full architecture.

# RELATABLE EXAMPLE

- What's your dream house like?
- Only saying *"a two-storey house"* conveys the structure but nothing else.

Two-story house structure (credit: chatGPT)

# ARCHITECTURAL STYLE

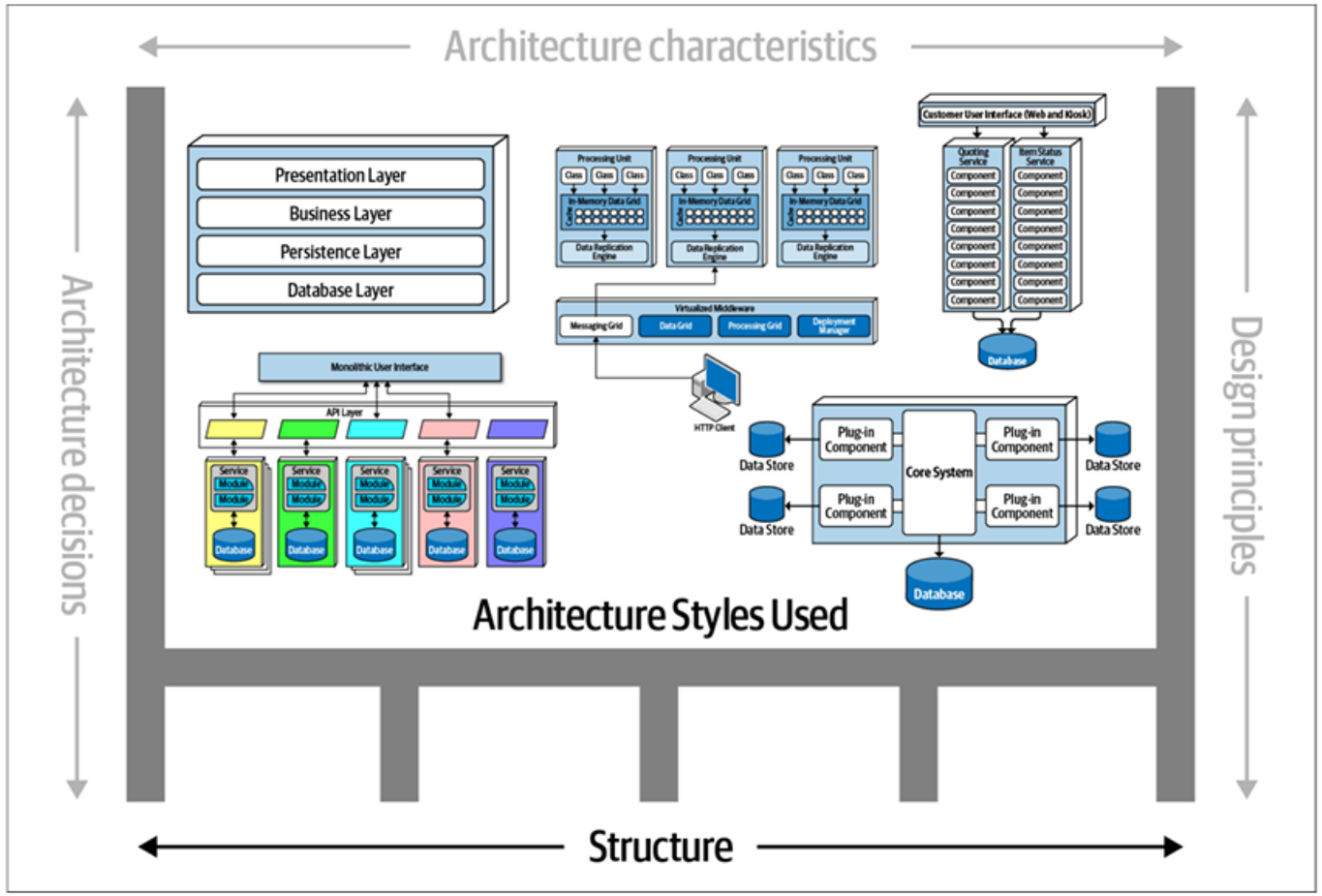



*Figure 1-3. Structure refers to the type of architecture styles used in the system*

# QUALITY ATTRIBUTES

- **Quality Attributes** (Architecture characteristics) are how we judge whether a system is good or bad, beyond *"does it work"*?
- They define the success criteria of a system.
  - These characteristics are *orthogonal* to functionality.
  - They do not depend on knowing what the system does, yet are essential for the system to function effectively.
    - You can discuss architecture characteristics without knowing what the system actually does.

# RELATABLE EXAMPLE

- Both houses satisfy the same functional requirement of two storeys. Only one satisfies the quality attribute of safety.

Two-story house structures (credit: chatGPT)

# QUALITY ATTRIBUTES

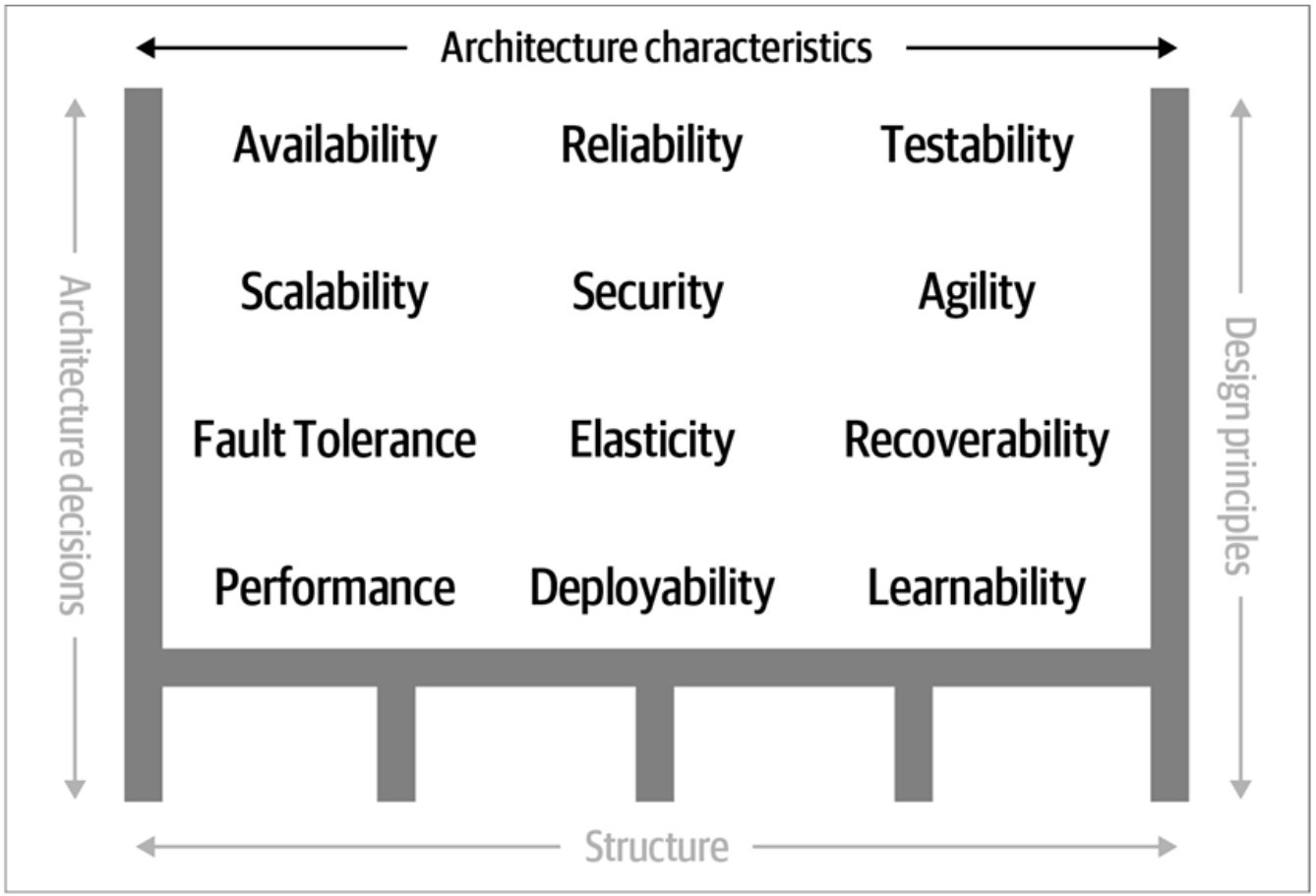



*Figure 1-4. Architecture characteristics refers to the "-ilities" that the system must support*

# ARCHITECTURE DECISIONS

- **Architecture decisions** specify rules for how the system is constructed.
- Such decisions impose constraints on the system.
- They guide development teams on what is allowed and what is not.
- Example - In a layered architecture, only business/service layers may access the database.
- If an architecture decision cannot be followed, it may be formally waived via a *variance*, usually approved by an architecture review board/ chief architect.

# RELATABLE EXAMPLE

- All load-bearing structures must be built using reinforced concrete, while non-load-bearing walls may use brick to optimize cost.

Concrete structure and Brick walls (credit: chatGPT)

# ARCHITECTURE DECISIONS

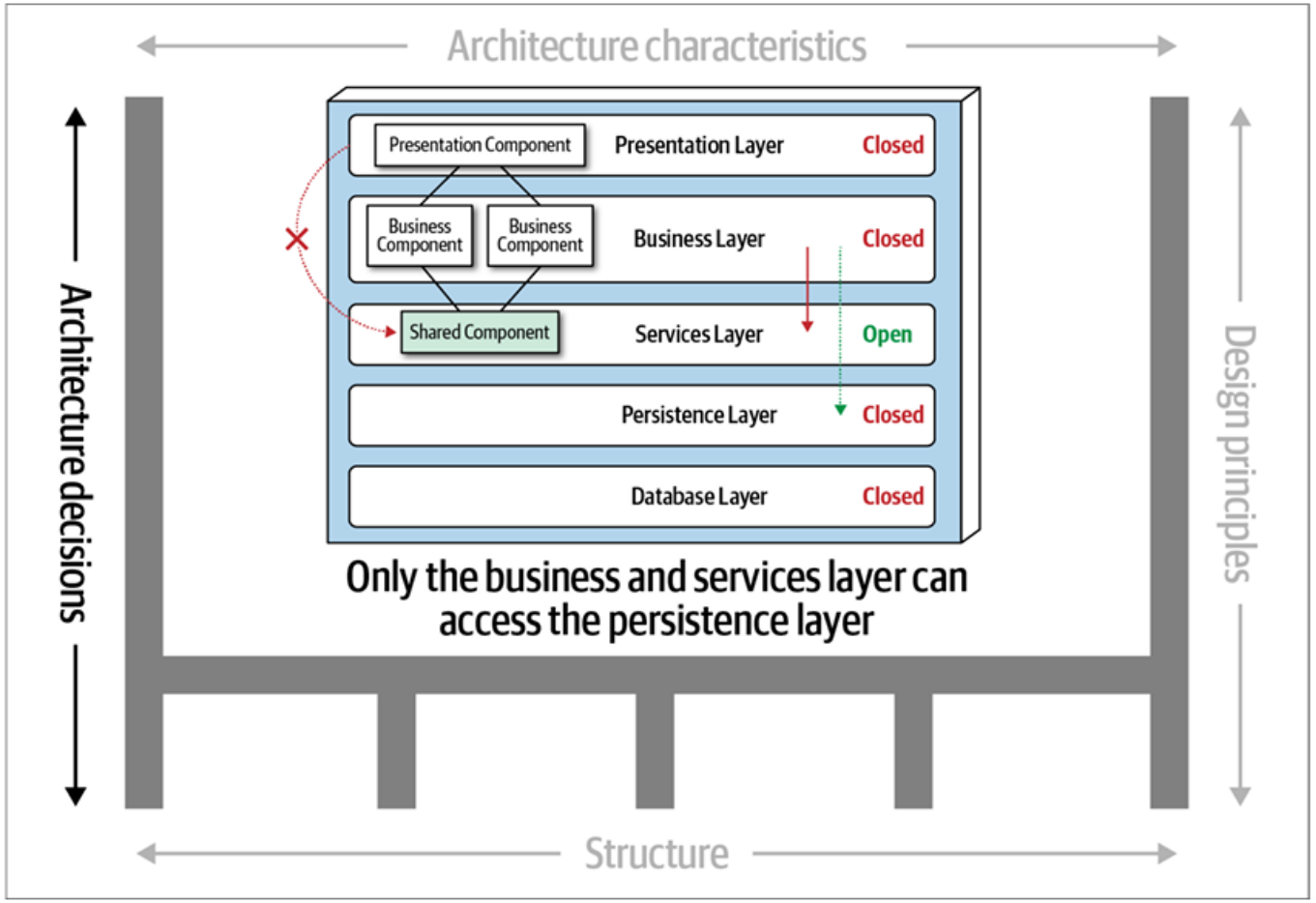



*Figure 1-5. Architecture decisions are rules for constructing systems*

# DESIGN PRINCIPLES

- Design principles are the final factor in defining software architecture.
- They act as guidelines, not strict rules.
- Unlike architecture decisions, they allow flexibility in implementation.
- Design principles guide developers' choices when rules cannot cover every situation.
  - Example - Prefer asynchronous messaging between microservices to improve performance.

# DESIGN PRINCIPLES

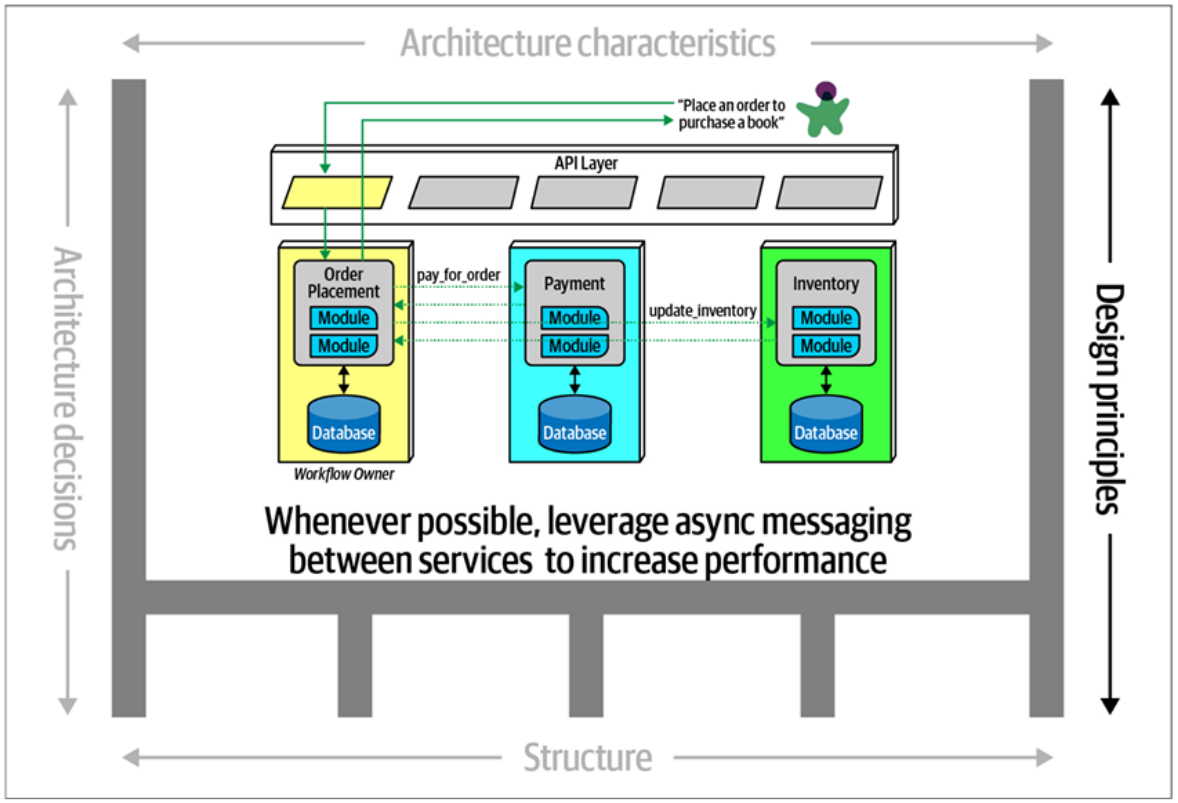



*Figure 1-6. Design principles are guidelines for constructing systems*

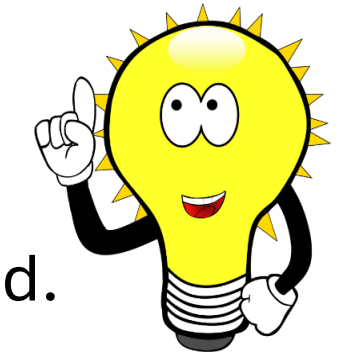
# RELATABLE EXAMPLE

- Prefer smooth flooring materials to improve comfort. However, select floor materials based on usage, prioritizing safety and aesthetics.

Concrete structure and Brick walls (credit: chatGPT)

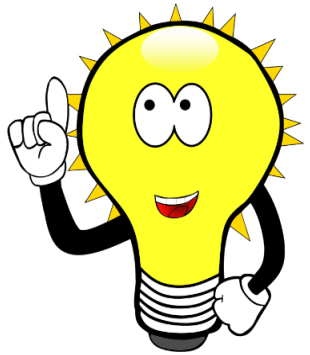
# WHAT IS A SOFTWARE ARCHITECT?

- The role of the software architect carries a broad and expanding scope of responsibility.
- A decade ago, architects focused mainly on technical concerns such as:
  - Modularity
  - Components
  - Design patterns
  - With the emergence of modern architectural styles (e.g.,microservices), this role has significantly expanded.



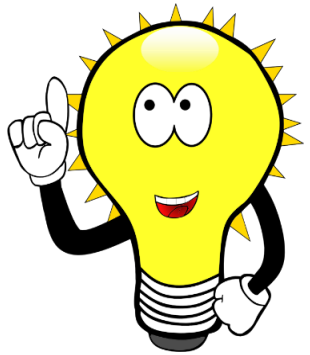
# WHAT IS A SOFTWARE ARCHITECT?

- Today, software architects must address a wider range of system-level concerns, beyond purely technical design.
- It can range from expert programmer up to defining the strategic technical direction for the company.
- It's not practical to exactly define the bounds of the role.
- Instead, however, we can clearly define the **expectations of an architect**.

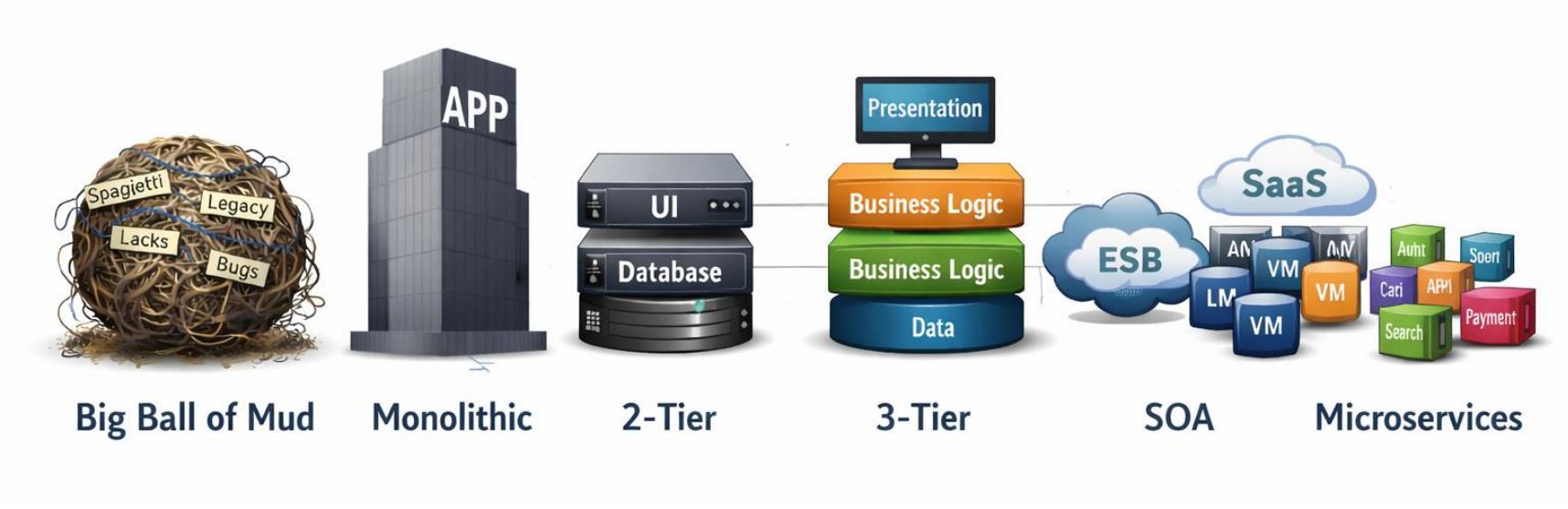


# EXPECTATIONS OF AN ARCHITECT

- Make architecture decisions
- Continually analyze the architecture
- Keep current with latest trends
- Ensure compliance with decisions
- Diverse exposure and experience
- Have business domain knowledge
- Possess interpersonal skills
- Understand and navigate politics



# EVOLUTION OF ARCHITECTURAL STYLES




Evolution of architectural styles (credit: chatGPT)

# SUMMARY

- Software architecture is a combination of architectural styles, quality attributes, architectural decisions and principles.
- Architectural choices are driven by quality attributes and constraints, not features (functional requirements) alone.
- Architectures evolve over time as systems, technologies, and environments change.
- The role of an architect is broadening with time.
- However, the core expectations of an architect remain largely unchanged.

# REFERENCES & FURTHER READING

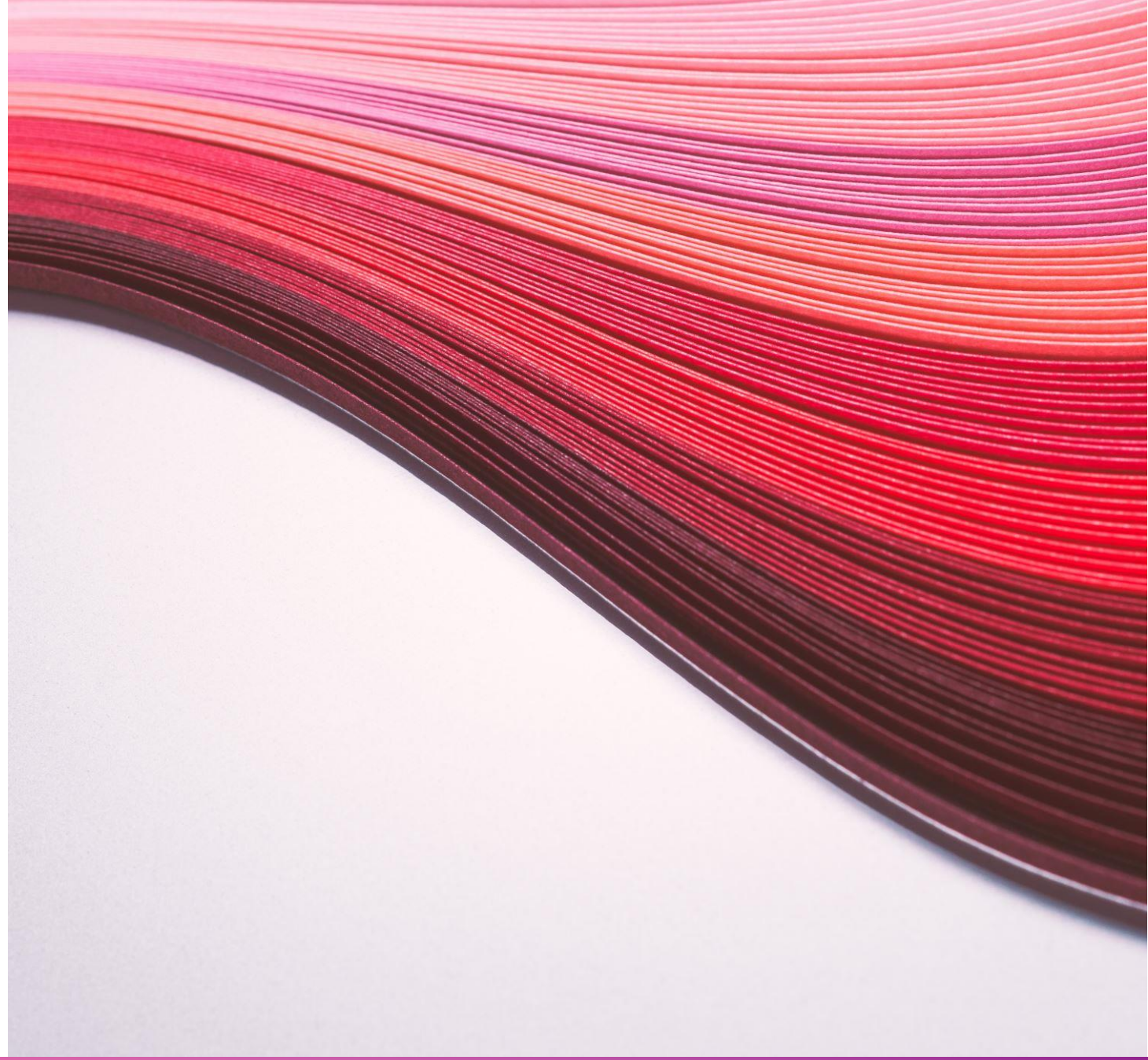
- Richards, Mark, & Ford, Neal, *"Fundamentals of Software Architecture: An Engineering Approach"* - Read chapter 01.
- Evolution of Software Architecture: From Mainframes and Monoliths to Distributed Computing

# ACKNOWLEDGEMENTS


- Software Architecture definition is from Richards, Mark, & Ford, Neal, *"Fundamentals of Software Architecture: An Engineering Approach"*.

# THANK YOU!

*Vishan Jayasinghearachchi*

*Department of Software Engineering*

*vishan.j@sliit.lk*